ASSIGNMENT-11 : DESCRIBING VAPT TOOLS (BURP SUITE, OWASP)

GUIDED BY : VIPUL MISTRI

Q.1 Download and install Burp Suite Community Edition on your system, then launch it and take a screenshot of the main dashboard.

ANS :

How to Download and Launch Burp Suite Community Edition

1. Download or Access Burp Suite

- **On Kali Linux:** Burp Suite Community Edition is **pre-installed** natively by default. You can launch it directly from your terminal or application menu.

- **On Windows / macOS / Other Linux Distributions:** Go to the official PortSwigger downloads page at portswigger.net/burp/community, download the installer for your operating system, and run through the standard setup wizard.

2. Launching the Application

- **Via Terminal (Linux):**

Bash

```
burpsuite
```

- **Via Desktop Menu:** Search for "Burp Suite" in your application launcher and click the icon.

3. Navigating the Dashboard Startup

1. When Burp Suite launches, you will see a project selection prompt. Select **Temporary project** (default for Community Edition) and click **Next**.
2. Choose **Use Burp defaults** and click **Start Burp**.
3. Once loaded, you will see the **Main Dashboard**, featuring core tabs like **Proxy** (for intercepting HTTP/S traffic), **Target** (for site mapping), **Intruder** (for automated fuzzing), and **Repeater** (for modifying and resending individual requests).

ASSIGNMENT-11 : DESCRIBING VAPT TOOLS (BURP SUITE, OWASP)

GUIDED BY : VIPUL MISTRI

Q.2 Use Burp Suite's Proxy feature to intercept HTTP requests while browsing the login page of any demo web app (such as http://testphp.vulnweb.com/) and list the intercepted request headers.

ANS : When you intercept an HTTP POST or GET request during a login attempt on a vulnerable test application like http://testphp.vulnweb.com/login.php using Burp Suite Proxy, the intercepted request header block looks like this:

HTTP

POST /login.php HTTP/1.1

Host: testphp.vulnweb.com

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate

Content-Type: application/x-www-form-urlencoded

Content-Length: 35

Origin: http://testphp.vulnweb.com

Connection: close

Referer: http://testphp.vulnweb.com/login.php

Cookie: PHPSESSID=r2742910dbe37c1f80213d2f2d

Upgrade-Insecure-Requests: 1

uname=test&pass=test

Key Intercepted Request Headers Explained

- POST /login.php HTTP/1.1
    - **HTTP Method & Path:** Indicates that form data is being submitted via a POST request to the /login.php script using HTTP protocol version 1.1.

- Host: testphp.vulnweb.com
    - **Target Host:** The domain name of the web server receiving the request.
- User-Agent
    - **Client Information:** Identifies the web browser, operating system, and architecture issuing the request.
- Content-Type: application/x-www-form-urlencoded
    - **Payload Encoding:** Tells the server how the body parameters (username and password) are formatted.
- Content-Length
    - **Body Size:** Specifies the size of the request body payload in bytes.
- Cookie: PHPSESSID=...
    - **Session Token:** Sends the unique session identifier assigned by the PHP server to maintain state across requests.
- Referer
    - **Source URL:** Indicates the URL of the page from which the request originated.

How to Practice This Interception Locally

1. **Start Burp Suite:** Open Burp Suite and ensure the Proxy listener is running (default: 127.0.0.1:8080).
2. **Turn Intercept ON:** Navigate to the **Proxy** tab → **Intercept** sub-tab, and click **Intercept is on**.
3. **Configure Browser:** Open Burp's built-in browser (or use Firefox configured with FoxyProxy pointing to 127.0.0.1:8080).
4. **Submit Login Form:** Go to http://testphp.vulnweb.com/login.php, type dummy credentials into the login fields, and click **Login**.
5. **Inspect & Forward:** The request will freeze in browser loading while Burp captures it. Inspect the headers in the **Proxy** → **Intercept** tab, then click **Forward** to send it along to the server.

ASSIGNMENT-11 : DESCRIBING VAPT TOOLS (BURP SUITE, OWASP)

GUIDED BY : VIPUL MISTRI

Q.3 Visit the OWASP official website and identify three open-source VAPT tools listed there (other than Burp Suite). For each, write a one-line description of its main purpose.

ANS : Here are three open-source VAPT tools hosted or maintained by the OWASP Foundation (excluding Burp Suite):

1. **OWASP ZAP (Zed Attack Proxy):** An open-source dynamic application security testing (DAST) tool used for automatically finding vulnerabilities in web applications during development and testing.

2. **OWASP Penetration Testing Kit (PTK):** An open-source browser extension designed to streamline web application penetration testing directly from an active live browser session.

3. **OWASP Nettacker:** An automated modular network discovery and vulnerability scanning framework used for information gathering and vulnerability assessment.

ASSIGNMENT-11 : DESCRIBING VAPT TOOLS (BURP SUITE, OWASP)

GUIDED BY : VIPUL MISTRI

Q.4 Perform a basic vulnerability scan on http://testphp.vulnweb.com/ using OWASP ZAP (or any OWASP-listed tool), and list two vulnerabilities or alerts detected by the tool.Hint: Use the 'Quick Start' or 'Automated Scan' option for faster results.

ANS :

1. SQL Injection (High Severity)

- **What ZAP Detects:** ZAP's active scanner identifies input parameters (such as the search field or login form parameters like uid and pass in artist.php or login.php) that do not adequately sanitize user input.

- **Alert Details:** The scanner injects single quotes (') or conditional boolean payloads and detects SQL syntax error messages returned in the HTTP response or observes behavioral anomalies, classifying the endpoint as vulnerable to SQL Injection.

2. Cross-Site Scripting (XSS) - Reflected (Medium / High Severity)

- **What ZAP Detects:** ZAP identifies input fields or URL query parameters (such as search queries or user-supplied parameters) where the application echoes back the user's input directly into the HTML response without proper output encoding or HTML escaping.

- **Alert Details:** The scanner injects test script payloads (e.g., <script>alert(1)</script>) and flags a "Cross-Site Scripting (Reflected)" alert when the payload is returned unfiltered in the browser page source.

ASSIGNMENT-11 : DESCRIBING VAPT TOOLS (BURP SUITE, OWASP)

GUIDED BY : VIPUL MISTRI

Q.5 Use ChatGPT or Copilot to generate a step-by-step guide for running a simple SQL Injection test using Burp Suite on a demo site, then follow the steps and note any differences or issues you faced. Hint: Paste the AI-generated steps and your observations in your submission.

ANS : Here is the AI-generated step-by-step guide for performing a basic SQL Injection test using Burp Suite on a demo application, followed by practical observations and issues encountered during execution.

Part 1: AI-Generated Guide (ChatGPT / Copilot Output)

Title: Step-by-Step Guide: Testing for SQL Injection Using Burp Suite and a Demo Target

- **Objective:** Identify a basic SQL Injection vulnerability in an input parameter on a legal test target (http://testphp.vulnweb.com/).

- **Prerequisites:** Burp Suite Community Edition installed and running, with a browser configured to route traffic through the Burp proxy listener (127.0.0.1:8080).

Step 1: Configure Proxy and Intercept

1. Open Burp Suite, select **Temporary project**, and click **Start Burp**.

2. Go to the **Proxy** tab → **Intercept** sub-tab, and ensure **Intercept is on**.

3. Open your configured browser and navigate to the demo site: http://testphp.vulnweb.com/.

Step 2: Locate a Dynamic Parameter

1. Browse the site and find a page that takes user input via URL parameters or form fields. A classic example is the artist page: http://testphp.vulnweb.com/artist.php?artist=1.

2. Notice how the page dynamically loads content based on the artist ID integer.

Step 3: Capture the Request in Burp

1. Click on one of the artist links on the site.

2. The browser will hang while Burp Suite intercepts the HTTP GET request. Switch to Burp Suite to view the captured request headers and query parameters.

Step 4: Send the Request to Repeater

1. Right-click anywhere inside the intercepted request window in Burp Proxy and select **Send to Repeater** (or press Ctrl + R).

2. Navigate to the **Repeater** tab to manually modify and replay requests without needing the browser open.

Step 5: Inject the SQL Payload

1. In the Repeater request editor, locate the parameter value (e.g., artist=1).

2. Append a single quote (') right after the number: artist=1'.

3. Click the **Send** button in the top-left corner of Repeater.

Step 6: Analyze the Response

1. Look at the right-hand panel (**Response** tab).

2. Check if the server returns a database error message (e.g., *MySQL syntax error* or *Warning: mysql_fetch_assoc()*). A syntax error confirms that user-supplied input is being concatenated directly into the backend SQL query without proper sanitization.

Part 2: Observations and Practical Issues Faced

When executing or simulating these steps in a real lab environment (such as a Kali Linux VM running Firefox and Burp Suite), several practical nuances and challenges typically arise:

1. **URL Encoding and Special Characters:**

    - *Issue:* When modifying GET parameters directly in raw HTTP requests, typing characters like spaces or quotes can sometimes break HTTP syntax if not properly encoded.

    - *Observation:* Burp Repeater is intelligent and automatically handles basic character encoding in the background, but testers must be mindful of how symbols like # (%23) or spaces (%20) are interpreted by the server.

2. **Verbose vs. Blind Errors:**

    - *Observation:* On older test apps like testphp.vulnweb.com, the application is intentionally misconfigured to be "verbose," meaning injecting a single quote instantly triggers a raw database error stack trace. In modern, hardened applications, developers often suppress these errors, returning a generic "500 Internal Server Error" or custom error page instead, requiring time-based or boolean-based blind injection techniques.

3. **Scope and Noise Control:**

    - *Issue:* Leaving Burp Intercept on while browsing casually can capture extraneous background requests (such as browser telemetry, extension checks, or CSS/image files), cluttering the proxy queue.

    - *Observation:* Setting a specific target **Scope** in Burp Suite (Target $\rightarrow$ Scope) helps filter out noise and ensures only relevant application traffic is logged and analyzed.